

Operating Systems Laboratory

Windows Student Guide: WSL 2 + Ubuntu + GCC

Complete installation, configuration, verification and troubleshooting guide

Purpose: Set up a Linux/Ubuntu environment on Windows for C programming and Operating Systems practicals such as process creation, system calls and CPU scheduling programs.

Item	Recommended
Windows	Windows 11 or Windows 10 version 2004 / Build 19041 or later
Linux environment	WSL 2
Distribution	Ubuntu
Compiler	GCC via Ubuntu package manager
Working folder	Ubuntu home directory, e.g. ~/OS
Administrator rights	Required for initial Windows installation

1. Before You Start

WSL (Windows Subsystem for Linux) provides a Linux environment directly inside Windows without requiring a traditional dual-boot installation or a full virtual machine. Microsoft's current installation command enables the required components, installs WSL 2, and installs Ubuntu by default.

Official prerequisites: Windows 10 version 2004 (Build 19041) or later, or Windows 11. Keep Windows updated and make sure you have enough free disk space.

Do not start with GCC. First make Ubuntu work, then verify WSL 2, then install GCC.

2. Check Windows Version

Press Windows + R → type winver → Enter. Record the Windows version/build. If the computer is older than the supported version, update Windows before continuing.

winver

3. Check Virtualization

Press Ctrl + Shift + Esc → Performance → CPU. Look for Virtualization: Enabled. If it says Disabled, WSL 2 may fail. Restart the computer, enter BIOS/UEFI, enable Intel VT-x / Intel Virtualization Technology or AMD SVM / AMD-V, save and restart.

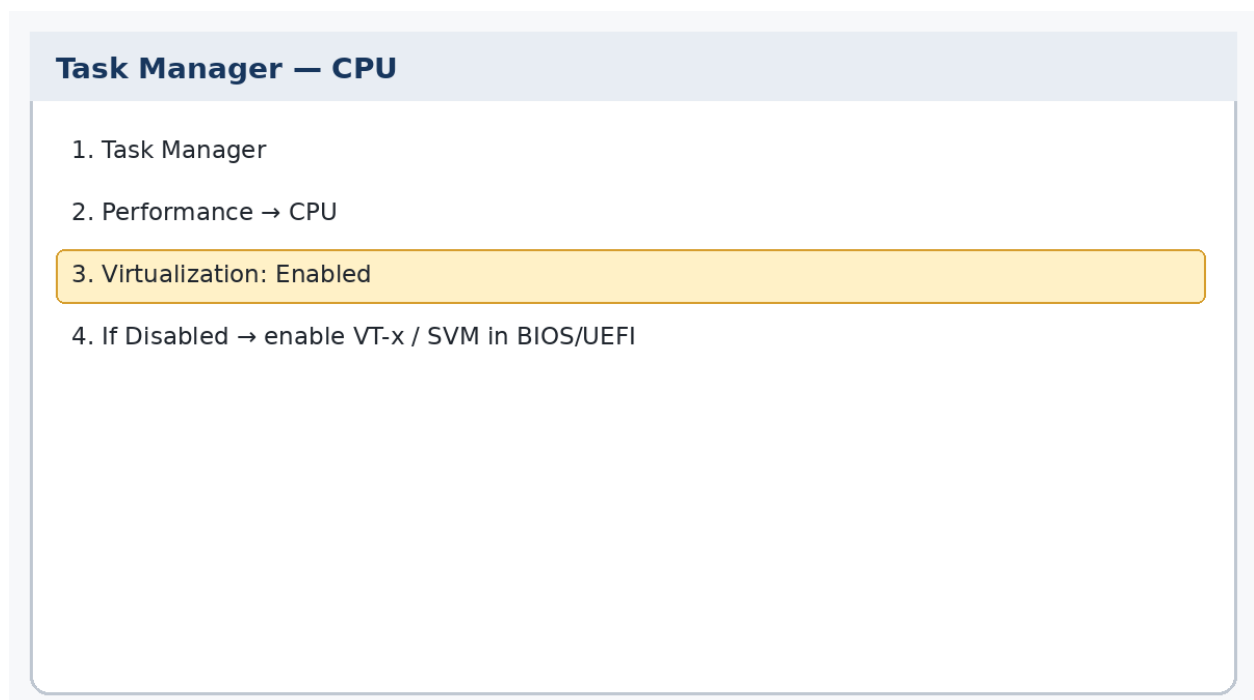


Figure: Task Manager CPU page showing virtualization enabled. The exact UI may vary slightly by Windows/macOS/UTM version.

4. Open PowerShell as Administrator

Use the Start menu, search for PowerShell, right-click Windows PowerShell (or PowerShell) and choose Run as administrator. A normal PowerShell window may not have the privileges required for the initial setup.

PowerShell — Run as administrator

1. Start menu
2. Search: PowerShell
3. Right-click Windows PowerShell
4. Select: Run as administrator

Figure: Opening PowerShell with administrator privileges. The exact UI may vary slightly by Windows/macOS/UTM version.

5. Install WSL and Ubuntu — Recommended Method

In the Administrator PowerShell window, run:

```
wsl --install
```

Restart Windows when prompted. Microsoft states that this command enables WSL and Virtual Machine Platform, installs the Linux kernel, sets WSL 2 as the default, and installs Ubuntu.

PowerShell — WSL installation

PS C:\Users\Student>

2. wsl --install

3. Installing: WSL + Virtual Machine Platform
4. Ubuntu will be installed
5. Restart Windows when prompted

Figure: Running the official WSL installation command. The exact UI may vary slightly by Windows/macOS/UTM version.

6. First Launch of Ubuntu

After restarting, open Start → Ubuntu. The first launch may take a few minutes while Ubuntu is initialized. Create a Linux username and password. The password is not displayed while you type; this is normal Linux behavior.

Ubuntu — first launch

1. Installing, this may take a few minutes...
2. Enter new UNIX username:
3. Enter new UNIX password:
4. Password typing is invisible — this is normal

Figure: Ubuntu first launch and creation of Linux user credentials. The exact UI may vary slightly by Windows/macOS/UTM version.

7. Verify That WSL 2 Is Installed

Open PowerShell and run:

```
wsl -l -v
```

Expected result should show Ubuntu with VERSION 2. You can also run:

```
wsl --status
```

If Ubuntu shows VERSION 1, convert it with:

```
wsl --set-version Ubuntu 2
```

Then verify again with `wsl -l -v`.

Verify WSL 2

PS C:\Users\Student>

2. `wsl -l -v`

3. NAME	STATE	VERSION
---------	-------	---------

4. Ubuntu	Running	2
-----------	---------	---

Figure: Checking the installed Ubuntu distribution and WSL version. The exact UI may vary slightly by Windows/macOS/UTM version.

8. How to Open Ubuntu Later

- Start menu → search Ubuntu → open Ubuntu.
- From PowerShell, type: `wsl`
- If multiple distributions exist, use: `wsl -d Ubuntu`

For OS practicals, students should normally work inside the Ubuntu terminal rather than in PowerShell.

9. Correct Linux Working Directory

After Ubuntu opens, run:

```
cd ~  
pwd
```

You should see a path similar to `/home/student`. Create an OS folder:

```
mkdir ~/OS  
cd ~/OS  
pwd
```

Keep Linux source files in the Linux filesystem (for example `~/OS`) rather than in protected Windows system folders. This avoids many permission and performance problems.

Ubuntu terminal — correct Linux home

1. student@pc:~\$
2. pwd
3. /home/student
4. cd ~
5. mkdir OS && cd OS

Figure: Using the Ubuntu home directory for OS laboratory programs. The exact UI may vary slightly by Windows/macOS/UTM version.

10. Understanding the C: Drive Permission Problem

A common mistake is trying to use Windows syntax inside Ubuntu, such as `cd C:` or `cd C:\Users\Student`. In WSL, Windows drives are exposed under `/mnt`.

Therefore:

```
cd /mnt/c
ls
```

C: is `/mnt/c`, D: is usually `/mnt/d`, and so on. If a Windows folder gives Permission denied, do not change permissions blindly. Move your Linux program into `~/OS` and work there. Protected folders such as Windows, Program Files and other security-controlled locations may restrict access.

C: drive inside WSL

1. `student@pc:~$`
2. `cd /mnt/c`
3. `ls`
4. `Users Windows Program Files`

Figure: Accessing the Windows C: drive correctly from WSL. The exact UI may vary slightly by Windows/macOS/UTM version.

11. If the Ubuntu Terminal Opens in C:\Windows\System32

Simply return to the Linux home directory:

```
cd ~
pwd
```

Then continue working in `~/OS`.

12. Update Ubuntu Package Information

```
sudo apt update
```

Enter the Ubuntu password when requested. If the password appears invisible while typing, continue typing and press Enter.

13. Install GCC

Install the standard build tools inside Ubuntu — not in Windows PowerShell:

```
sudo apt install build-essential
```

Press Y when asked. Then verify:

```
gcc --version
make --version
```

Microsoft's Ubuntu tutorial recommends updating the package list first and then installing software with apt.

GCC installation

1. student@pc:~\$
2. **sudo apt update**
3. **sudo apt install build-essential**
4. gcc --version

Figure: Installing build-essential and checking GCC. The exact UI may vary slightly by Windows/macOS/UTM version.

14. First C Program — Complete Test

Move to the OS folder:

```
cd ~/OS
```

Create hello.c:

```
nano hello.c
```

Enter:

```
#include <stdio.h>

int main()
{
    printf("Hello from Ubuntu WSL\n");
    return 0;
}
```

Save with Ctrl+O → Enter. Exit with Ctrl+X.

Compile and run:

```
gcc hello.c -o hello
./hello
```


Compile and execute C program

1. `gcc hello.c -o hello`
2. `./hello`
3. Hello from Ubuntu WSL

Figure: Compiling and executing the first C program. The exact UI may vary slightly by Windows/macOS/UTM version.

15. Common Problem: 'Forbidden' / Microsoft Store Download Failure

If the student sees a Forbidden/403 error, Microsoft Store download failure, or the installation hangs, first make sure the internet connection is working and Windows is updated. Then try:

```
wsl --update
```

```
wsl --install --web-download -d Ubuntu
```

The web-download option is an official Microsoft-supported alternative when the normal distribution download is not progressing. If the college network blocks the Microsoft Store or related downloads, the manual installation route may be required.

Do not download random Ubuntu packages from unofficial websites.

Forbidden / Store download workaround

PS C:\Users\Student>

2. wsl --update

3. wsl --install --web-download -d Ubuntu

4. If still blocked: use manual installation route

Figure: Web-download workaround for WSL distribution installation. The exact UI may vary slightly by Windows/macOS/UTM version.

16. Common Error: 0x8007019e

Meaning: the Windows Subsystem for Linux optional feature is not enabled. Open 'Turn Windows features on or off' and ensure Windows Subsystem for Linux and Virtual Machine Platform are enabled. Restart and try again.

17. Common Error: 0x80370102

Usually check: (1) Virtual Machine Platform is enabled, (2) CPU virtualization is enabled in BIOS/UEFI, (3) Windows is updated. Then run `wsl --update`, restart WSL with `wsl --shutdown`, and try Ubuntu again.

Common WSL errors

1. 0x8007019e → enable WSL feature
2. 0x80370102 → enable virtualization
3. Permission denied → work under ~
4. GCC package error → sudo apt update first

Figure: Quick mapping of common WSL installation errors to their fixes. The exact UI may vary slightly by Windows/macOS/UTM version.

18. GCC Troubleshooting

Problem	What to do
Unable to locate package build-essential	Run <code>sudo apt update</code> , then <code>sudo apt install build-essential</code>
Permission denied during apt install	Use <code>sudo</code> for package installation; do not use <code>chmod 777</code>
Could not get dpkg/apt lock	Wait for another update/install process to finish; do not delete lock files blindly
gcc: command not found	Install build-essential and run <code>gcc --version</code>
Compilation permission denied	Work in <code>~/OS</code> ; check that the source file is in the Linux filesystem
Program will not execute	Compile with <code>gcc file.c -o file</code> and run <code>./file</code>

19. Recommended Windows Settings for the Class

Setting	Required/Recommended
Windows version	Windows 11 or supported Windows 10
Virtualization	Enabled
Windows Subsystem for Linux	Enabled
Virtual Machine Platform	Enabled
WSL version	2
Ubuntu	Installed and verified
GCC	Installed inside Ubuntu
OS source folder	<code>~/OS</code>

20. Verification Checklist Before the Lab

- Windows version checked
- Virtualization = Enabled
- `wsl --install` completed
- Computer restarted
- Ubuntu opens
- Linux username/password created
- `wsl -l -v` shows VERSION 2
- `cd ~` works
- `mkdir ~/OS` works
- `sudo apt update` works
- build-essential installed
- `gcc --version` works
- `hello.c` compiles
- `./hello` executes successfully

21. Quick Recovery Sequence

If WSL behaves unexpectedly, run these commands in Administrator PowerShell:

```
wsl --shutdown
wsl --update
wsl -l -v
wsl
```

Then inside Ubuntu:

```
cd ~
sudo apt update
sudo apt install build-essential
gcc --version
```

22. OS Laboratory Final Test

Students should be able to complete all of the following before attending the practical:

```
mkdir -p ~/OS
cd ~/OS
nano hello.c
gcc hello.c -o hello
./hello
```

Once this test succeeds, the Ubuntu/GCC environment is ready for OS programs such as `fork()`, `exec()`, process creation, system calls, scheduling algorithms and related C practicals.

Official References

Official reference: Microsoft Learn — Install WSL: <https://learn.microsoft.com/en-us/windows/wsl/install>

Official reference: Microsoft Learn — Manual WSL installation:

<https://learn.microsoft.com/en-us/windows/wsl/install-manual>

Official reference: Microsoft Learn — Troubleshooting WSL:

<https://learn.microsoft.com/en-us/windows/wsl/troubleshooting>

Official reference: Microsoft Learn — Getting started with Linux using WSL:

<https://learn.microsoft.com/en-us/windows/wsl/tutorials/linux>

Prepared as a student installation guide for Operating Systems laboratory use.